



NORTH SOUTH UNIVERSITY

Department of Electrical and Computer Engineering

PROJECT REPORT

Enhancing Process Monitoring, System Call Tracing, and Priority Scheduling in xv6

Course: CSE323 (Operating System)

Section: 04

Submitted to : Farhan Tanvir Khan (FTH)

Date: 12-05-2026

Submitted by: Trio-tech

Name	ID	Feature
Kamrun Nahar Majumder Kakon	2211801642	Process Monitoring
Maimuna Islam Muna	2311818042	Priority-Based Scheduling
Farhana Early	2312602042	System Call Tracing

Abstract

xv6 is a small Unix-like educational operating system developed at MIT and widely used in undergraduate operating systems courses to teach fundamental concepts such as process management, system calls, memory management, and CPU scheduling. Although its minimal design makes it highly suitable for academic learning, xv6 lacks many of the monitoring, scheduling, and debugging facilities available in modern operating systems.

This report presents the design, implementation, and testing of three major enhancements added to the xv6 kernel to improve its observability, scheduling efficiency, and debugging capabilities. The first enhancement, **Process Monitoring**, extends the process structure by introducing `run_count` and `start_ticks` fields and implements a new `pinfo()` system call along with a user-space `ps` utility that displays formatted process information, scheduling statistics, and process lifetime details.

The second enhancement, **Priority-Based Scheduling with Aging Mechanism**, replaces xv6's default round-robin scheduler with a priority-driven scheduling mechanism. Each process is assigned a default priority value of 10, and the scheduler always selects the `RUNNABLE` process with the lowest priority value for execution. To prevent starvation, an aging mechanism dynamically improves the priority of processes that remain waiting for more than 50 ticks.

The third enhancement, **System Call Tracing**, introduces a `trace_mask` field into the process structure and modifies the kernel's system call dispatcher to trace selected system calls. Whenever a system call corresponding to a set bit in the mask is invoked, the kernel logs the system call name and its return value, enabling easier debugging and analysis of process behavior.

All three features were successfully integrated into a single xv6 kernel build and validated through functional testing. These enhancements transform xv6 into a more transparent and educationally effective operating system, providing students with practical experience in kernel modification, scheduling algorithms, process analysis, and low-level systems programming.

Introduction

An operating system is the software layer that sits between user applications and computer hardware. It manages processes, memory, files, and devices while ensuring that multiple programs can run safely and efficiently. Understanding these low-level mechanisms is an essential part of computer science and engineering education.

xv6 is a small Unix-like teaching operating system originally developed at MIT. It was designed specifically for education, providing a simplified yet realistic implementation of important operating system concepts such as process management, scheduling, virtual memory, file systems, and system calls. Due to its compact and well-structured code base, xv6 is widely used in undergraduate operating systems courses around the world.

Although xv6 is highly effective as a learning platform, the default implementation intentionally omits several features commonly found in modern operating systems. The kernel provides limited visibility into running processes, relies on a basic round-robin scheduling policy, and lacks any built-in mechanism for tracing system calls. As a result, observing system behaviour, analyzing scheduling decisions, and debugging user programs become difficult for students working with the operating system.

This project addresses these limitations by extending xv6 with three important kernel-level features: process monitoring, priority-based scheduling with aging, and system call tracing. The process monitoring feature provides runtime information about active processes, including execution statistics and process relationships. The scheduling enhancement replaces the default round-robin scheduler with a priority-driven scheduler while introducing an aging mechanism to prevent starvation. In addition, a tracing mechanism is implemented to monitor selected system calls and display their return values for debugging and analysis purposes.

The project is implemented entirely within the xv6 kernel running on the x86 architecture using QEMU under WSL (Windows Subsystem for Linux). The primary goal of this work is educational: to provide practical experience with kernel modification, process management, scheduler design, and system call handling while improving xv6 as a platform for operating systems learning.

Literature Review

xv6 was originally developed for the MIT 6.828 course and later adapted to RISC-V systems as a compact Unix-like teaching operating system. Cox, Kaashoek, and Morris describe its design in detail, highlighting key kernel components such as the proc structure, system call interface, and a simple round-robin scheduler [1]. The primary design goal of xv6 is not feature completeness but conceptual clarity, making it a stable base for student-level kernel extensions.

In modern operating systems, process monitoring is typically achieved through structured interfaces that expose kernel-maintained process metadata. Linux, for example, implements the /proc virtual filesystem, where each process is represented as a directory containing files such as status, stat, and maps, providing detailed runtime and memory information [2]. Silberschatz et al. further formalize this concept through the Process Control Block (PCB), which includes scheduling state, register context, memory limits, and accounting information [3]. Compared to these systems, xv6 exposes only minimal process information, limiting runtime observability.

CPU scheduling has been extensively studied, with classical algorithms including First-Come First-Served (FCFS), Round Robin (RR), Shortest Job First (SJF), and Multilevel Feedback Queues (MLFQ), as described by Tanenbaum [4]. A key issue in priority-based systems is starvation, where low-priority processes may never execute. To address this, aging techniques were introduced, gradually increasing the priority of waiting processes over time [5]. While modern systems such as Linux's Completely Fair Scheduler (CFS) use virtual runtime instead of explicit priorities, aging remains widely used in educational and simplified schedulers due to its clarity and ease of implementation [3].

System call tracing is commonly implemented in Unix-like systems using tools such as strace, which intercept system calls via the ptrace() interface and records arguments and return values for debugging purposes [6]. In kernel-based implementations, tracing is typically integrated at the central system call dispatch layer, where all system calls pass through a single handler function. This approach is consistent with implementations in educational operating systems, including MIT's 6.S081 labs, where a bitmask-based tracing mechanism is often used to selectively log system calls [7].

Existing literature collectively shows that process monitoring is naturally supported through PCB-style structures, scheduling fairness can be improved using priority and aging mechanisms, and system call tracing is most effectively implemented at the syscall dispatch level. These established principles provide a clear foundation for extending xv6 with improved observability, scheduling, and debugging capabilities within its minimal kernel architecture.

Methodology

Development Environment

All development was performed on xv6, compiled with GCC and executed within the QEMU hardware emulator on WSL (Windows Subsystem for Linux). The make qemu command was used to compile and launch the system throughout development. Nine existing files were modified and two new user-space files were added across the code base.

Files Modified and Added:

File	Type	Change Summary
proc.h	Modified	Added priority, wait_ticks, run_count, start_ticks, trace_mask to struct proc; new struct pinfo
proc.c	Modified	allocproc(), fork(), scheduler() rewritten; new getpinfo() function
defs.h	Modified	Added prototype for getpinfo()
syscall.h	Modified	Added SYS_pinfo (22), SYS_setpriority (23), SYS_trace (24)
syscall.c	Modified	Added syscall_names array; modified dispatcher to check trace_mask
sysproc.c	Modified	Added sys_trace(), sys_setpriority(), sys_pinfo()
user.h	Modified	Added user-space prototypes for trace(), setpriority(), pinfo()
usys.S	Modified	Added assembly entry points for the three new system calls
Makefile	Modified	Added _ps and _trace to UPROGS; ps.c to EXTRA
ps.c	New (user)	Utility that calls pinfo() and prints formatted process table
trace.c	New (user)	Utility that sets trace mask and execs target command

Feature 1 : Process Monitoring (Kamrun Nahar Majumder Kakon)

Kernel Data Structure Changes

Two new fields were added to the proc structure in kernel/proc.h. The field run_count records how many times the scheduler has selected that process for execution. The field start_ticks records the system tick count at the moment the process was created. A new structure named struct pinfo was also defined. This structure acts as a data container for transferring process information from the kernel to user space. It stores the following fields for each active process:

- PID and parent PID (PPID)
- Process state (UNUSED, SLEEPING, RUNNABLE, RUNNING, or ZOMBIE)
- Process name and memory size (sz)
- run_count and start_ticks for scheduling statistics

- An array of child PIDs and a count of the number of children

Changes to proc.c

In kernel/proc.c, the allocproc() function was modified to initialise run_count to zero and to record the current system tick in start_ticks whenever a new process slot is allocated. The scheduler() function was updated to increment run_count each time the CPU switches to a process, so the counter always reflects how many times each process has been scheduled.

A new kernel function, getpinfo(), was implemented. It iterates through the entire process table and, for every active process, collects the PID, PPID, state, name, memory size, run_count, start_ticks, and the list of child PIDs. The collected data is written into an array of struct pinfo entries and returned to user space using copyout().

User-Space Program: ps.c

A new user program named ps.c was written and added to the xv6 build. When executed, ps invokes the pinfo() system call to retrieve a snapshot of all active processes and displays results as a formatted table. The column labelled Ticks Alive is calculated by subtracting start_ticks from the current tick count.

Feature 2 : Priority-Based Scheduling with Aging Mechanism (Maimuna Islam Muna)

Process Structure Changes

Two new integer fields were added to the proc structure in kernel/proc.h. The field priority stores the current priority value of the process, where a lower number indicates higher priority. The field wait_ticks counts the number of scheduler ticks that a process has spent in the RUNNABLE state without being selected. When a new process is created, allocproc() sets its default priority to 10.

Scheduler Algorithm

The scheduler loop in kernel/proc.c was rewritten to replace the round-robin approach with a priority-based selection strategy. On each scheduling pass, the scheduler searches the process table for all RUNNABLE processes and selects the one with the lowest priority value. The updated algorithm works as follows:

- The scheduler loops through the process table to find all RUNNABLE processes.
- It selects the process with the lowest priority value (most important).
- The selected process is given the CPU and its wait_ticks is reset to zero.
- For all other RUNNABLE processes that were not selected, wait_ticks is incremented.
- If any RUNNABLE process has been waiting for more than 50 ticks, the aging mechanism automatically decrements its priority value by one.

Aging Mechanism

The aging logic is applied inside the scheduler's search loop. Each time the scheduler runs and a process is not selected, its wait_ticks increases. When wait_ticks exceeds

the threshold of 50, the process's priority is reduced by one. This gradual improvement ensures that even a low-priority process will eventually reach a priority level that gets it scheduled, preventing indefinite starvation. Once the process is scheduled, its `wait_ticks` is reset to zero.

setpriority() System Call

A new system call, `sys_setpriority()`, was implemented in `kernel/sysproc.c`. This allows a user program to manually set the priority of any process by supplying a PID and a new priority value. The call validates the input and updates the `proc.priority` field under the process table lock.

Feature 3: System Call Tracing (Farhana Early)

Process Structure Change

A new field named `trace_mask` of type `uint64` was added to the `proc` structure in `kernel/proc.h`. Each bit in this mask corresponds to a system call number. When a particular bit is set, the kernel will log information about that system call every time the process invokes it.

Changes to syscall.c

A `syscall_names` array was added to `kernel/syscall.c`. This array maps each system call number to its human-readable name (for example, mapping number 1 to the string "fork"). The central `syscall()` dispatch function was then modified to check `trace_mask` after each system call completes. If the bit corresponding to the current system call number is set in the calling process's `trace_mask`, the kernel prints a trace message in the following format:

```
PID: <pid> | Syscall: <name> (number: <num>) | Returns: <return_value>
```

User Utility: trace.c

A new user program named `trace.c` was written. It accepts a numeric mask as its first argument and the name of a target command as the remaining arguments. The program calls the `trace()` system call to set the mask in its own process structure and then uses `exec()` to replace itself with the target command. From that point forward, every system call made by the target command that matches the mask is logged by the kernel.

Mask Inheritance via fork()

The `fork()` function in `kernel/proc.c` was modified to copy the `trace_mask` from the parent process to the newly created child process. This means that if a shell command is traced, all sub-processes it spawns will also be traced automatically, without any additional action from the user — mirroring how `strace` works on Linux.

Results and Findings

Process Monitoring : ps Command Output

The ps user program was tested against a set of processes with known states. The following output was observed when running ps on the live xv6 system:

```
$ ps
PID  PPID  STATE  SIZE  RUNS  TICKS  NAME  CHILDREN
-----
1    0    SLEEPING 12288  15    2834  init   2
2    1    SLEEPING 16384  20    2830  sh     3
3    2    RUNNING  32768  8      4     ps    none
```

In all test cases, the reported PID, PPID, process name, memory size, and run_count matched the expected values. The Ticks Alive column correctly reflected elapsed time by subtracting start_ticks from the current tick count. The child PID list accurately showed process hierarchy created by fork().

Test Case	Expected	Observed	Status
Valid PID query	Correct procinfo fields	Correct procinfo fields	PASS
PPID for child process	Parent PID returned	Parent PID returned	PASS
Child list after fork()	Child PIDs listed	Child PIDs listed	PASS
Memory size query	Correct sz value	Correct sz value	PASS
Invalid PID supplied	Return -1 (error)	Return -1 (error)	PASS

Priority-Based Scheduling: mytest Output

The priority scheduler was tested by launching concurrent processes with different priority values assigned using setpriority(). The following output was observed from the mytest program:

```
$ mytest
Starting priority scheduler test...
Child 1 (Priority 0) Finished!
Child 2 (Priority 10) Finished!
Child 3 (Priority 20) Finished!
All children finished. Test complete!
```

The process with the lowest priority number (Priority 0) consistently completed first, followed by Priority 10, then Priority 20 — confirming that the selection logic is correct. The aging mechanism was also validated: a low-priority process running alongside a CPU-hungry high-priority process eventually received CPU time after its wait_ticks exceeded the 50-tick aging threshold.

System Call Tracing : trace Output

System call tracing was tested by running the trace utility with mask 32 (bit 5 = read system call). The following output was observed when running trace 32 ls:

```
$ trace 32 ls
PID: 8 | Syscall: read (number: 5) | Returns: 16
.      1 1 512
PID: 8 | Syscall: read (number: 5) | Returns: 16
..     1 1 512
PID: 8 | Syscall: read (number: 5) | Returns: 16
README 2 2 2286
PID: 8 | Syscall: read (number: 5) | Returns: 16
cat     2 3 15444
PID: 8 | Syscall: read (number: 5) | Returns: 16
echo    2 4 14352
... (continues for each directory entry)
```

Only read system calls (bit 5) were logged, confirming the bitmask filter works correctly. When a full mask was used, all system calls were logged by name with their return values in the exact order made. The mask inheritance test confirmed that child processes produced trace output without needing to call trace() themselves.

Conclusion

This project successfully extended the xv6 kernel with three working features: process monitoring, priority-based scheduling with aging, and system call tracing. Each feature was implemented through targeted modifications to the kernel source files and validated through functional testing.

The process monitoring feature provides a clear and accessible view of all running processes. By adding run_count and start_ticks to the process structure and implementing the pinfo() system call, users can now observe scheduling behaviour and process lifetime through the ps command. The priority scheduler gives more important processes preference over less important ones while ensuring that no process is permanently denied CPU time, thanks to the 50-tick aging threshold. The system call tracer, built around a bitmask and inherited through fork(), gives users a direct view of the interaction between user programs and the kernel.

Each feature required meaningful engagement with core kernel subsystems, including process table management, the scheduler loop, system call dispatch, and process creation. The project therefore achieved both its technical and educational goals.

References

- [1] R. Cox, F. Kaashoek, and R. Morris, "xv6: a simple, Unix-like teaching operating system," MIT PDOS Lab, 2023. [Online]. Available: <https://pdos.csail.mit.edu/6.828/2023/xv6/book-riscv-rev3.pdf>
- [2] M. Kerrisk, The Linux Programming Interface: A Linux and UNIX System Programming Handbook. No Starch Press, 2010.
- [3] A. Silberschatz, P. B. Galvin, and G. Gagne, Operating System Concepts, 10th ed. Hoboken, NJ: Wiley, 2018.
- [4] A. S. Tanenbaum and H. Bos, Modern Operating Systems, 4th ed. Upper Saddle River, NJ: Pearson, 2015.
- [5] L. Kleinrock, Queueing Systems, Volume 2: Computer Applications. New York: Wiley-Interscience, 1976.
- [6] I. Wienand, "A brief introduction to strace," Linux Journal, 2012.
- [7] MIT PDOS, "Lab: Traps — xv6: Riscv," 6.S081 Operating System Engineering Labs, MIT, 2023. [Online]. Available: <https://pdos.csail.mit.edu/6.828/2023/labs/traps.html>
- [8] J. L. Peterson and A. Silberschatz, Operating System Concepts, 1st ed. Addison-Wesley, 1983.